



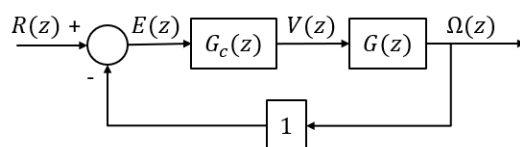
ENG 147: Mechatronics Lab 3: Digital Speed Control

Spring 2026

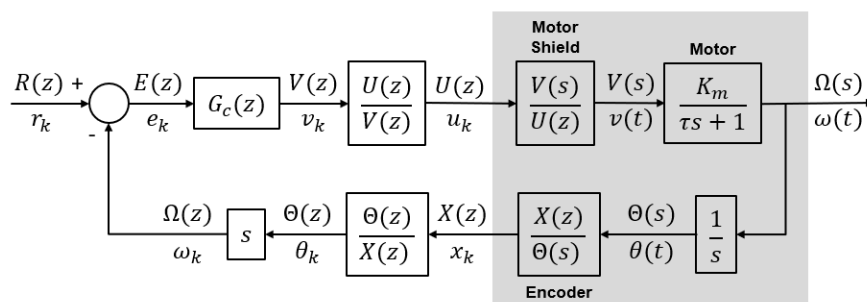
Objective: In this lab, you use the code framework you built in Lab 1 and Lab 2 to implement digital closed-loop speed control of your pancake motor. You will design and implement two different versions of the digital compensator $G_c(z)$: one using design by discrete equivalent, and the other using direct digital design.

With each design, you will attempt to meet **all** of the following specifications for a step input of 100 rad/s:

- Closed-loop damping ratio > 0.5 (you may wish to translate to a more measurable overshoot spec)
- Steady-state error $< 1\%$ of the reference
- Maximum absolute control effort $< V_{max}$, where V_{max} is the maximum motor voltage observed in Lab 1
- Minimize settling time (you will set your own settling time spec)



Theoretical Control Loop (discretized plant)



Actual Implementation with Hardware

1. Plug in your battery to ensure that it is charged. Leave your charger on the 1C setting.
2. If you have not completed Lab 1 or Lab 2 yet, please complete those assignments now as you will need the code and results from them to make progress on Lab 3.
3. In your lab writeup, clearly indicate the continuous time motor transfer function $G(s)$, whose constants K_m and τ you identified in Lab 1 or Lab 2, that you will use in simulation for this lab:

$G(s) =$ _____

Part 1: Design by Discrete Equivalent

4. Assuming that you will use a sampling time of $T_s = 10\text{ms}$ (ten milliseconds), select and justify an appropriate/achievable settling time specification. Hints: what is the Nyquist frequency associated with this **sampling** time? What open loop pole frequency can you expect to accommodate with reasonable accuracy? How do you expect this to affect closed loop poles and **settling** time?

Settling time target: _____

5. Use root locus methods to design a continuous-time compensator $G_c(s)$ that achieves the closed-loop performance specifications listed above.

$$G_c(s) = \underline{\hspace{2cm}}$$

Provide documentation (from MATLAB `step()`, `stepinfo()`, `damp()`, etc.) that your compensated system meets each of the performance specifications.

6. Use the MATLAB `c2d()` function with $T_s = 10\text{ms}$ and Tustin's rule to compute a discrete compensator $G_c(z)$ that is equivalent to your continuous time compensator $G_c(s)$.

$$G_c(z) = \frac{V(z)}{E(z)} = \underline{\hspace{2cm}} \quad T_s = 10\text{ms}$$

7. Using a sample time of $T_s = 10\text{ms}$, call `c2d()` in MATLAB with the zero-order hold approximation to find a discrete equivalent $G(z)$ of your plant $G(s)$.

$$G(s) = \underline{\hspace{2cm}} \quad G(z) = \underline{\hspace{2cm}} \quad T_s = 10\text{ms}$$

8. Provide documentation (from MATLAB `step()`, `stepinfo()`, `damp()`, etc.) that your **digital** closed-loop system meets the performance specifications **in simulation**.
9. **Compensator 1:** Write the difference equation corresponding to this transfer function. All subscripts of v and e should be $\leq k$ (your compensator doesn't have access to future values!):

$$v_k = \underline{\hspace{2cm}} \quad T_s = 10\text{ms}$$

10. Extend your Arduino code from Lab 2 to implement this digital control law following the “actual” block diagram above. You may wish at the top of your C++ file to `#define` several constants (e.g. A, B, C, etc.) that multiply samples of your discrete voltage and error signals. Also `#define` your sampling time T_s so that you **don't forget to update this value if you change your compensator**. Note that on the first pass(es) through your loop you may need to exclude terms that require previous values of the error or command voltage signals (e.g. e_{k-1} , v_{k-1} , etc.). Ensure that time-critical operations (reading encoder through writing PWM command) are performed efficiently at the start of each sample period. Your code should print at least the following data columns to the serial port for storage (e.g. in CSV format):

- Timestamp (e.g. from `micros()`)
- Speed in rad/s
- Error in rad/s
- Desired command voltage
- PWM value actually commanded to the motor

11. Plot your experimental speed and control effort responses to a 100rad/s step input on the same axes as the simulated step responses. You may wish to repeat your experiment, adjusting factors including your reference input value, your $\frac{U(z)}{V(z)}$ mapping function, and even your assumed motor constants K_m and τ to identify factors that improve or degrade the correspondence between your simulated and experimental step responses. Provide plots and parameter values for your final compensator, and demonstrate whether you did or did not meet the performance specifications. If you cannot meet these performance specifications, design your compensator to get as close as possible and explain the limitation(s) you encountered.

Part 2: Direct Digital Design

12. Using a sampling time of **at minimum** $T_s = 100\text{ms}$ (0.1 seconds), design a compensator in the digital domain to meet the performance specifications listed above.

$$G_c(z) = \frac{V(z)}{E(z)} = \underline{\hspace{2cm}} \quad T_s = \underline{\hspace{2cm}}$$

13. Discretize your plant with `c2d()` in MATLAB with the zero-order hold approximation, close the loop and provide documentation (from MATLAB `step()`, `stepinfo()`, `damp()`, etc.) that your digital closed-loop system meets the performance specifications.
14. **Compensator 2:** Write the difference equation corresponding to this transfer function. All subscripts of v and e should be $\leq k$ (your compensator doesn't have access to future values!):

$$v_k = \underline{\hspace{2cm}} \quad T_s = \underline{\hspace{2cm}}$$

15. Define in your Arduino code a set of constants **and the appropriate sampling time** that correspond to the difference equation for Compensator 2. You can comment out the constants and sampling time associated with the difference equation for Compensator 1.
16. Repeat the process of applying a reference speed step input and comparing the results to your simulation. This time you can experiment with sampling time as an additional parameter. How large a sampling time can you achieve? As before, please provide plots, parameter values, and demonstrate performance relative to the response specifications in your lab writeup.

Writeup Guidelines and Rubric: Your writeup should include all items and analyses specified above. Be sure to include a description and discussion of any changes you made to your compensator design or control loop in Steps 11 and 16. Don't forget to include your Arduino (C++) and MATLAB code in your PDF. Grades will be determined by assessing your combined writeup and in-person checkoff using the following rubric:

- (10 pts) Setting appropriate specifications (especially settling time and damping/peak overshoot)
- (20 pts) Design by discrete equivalent: Compensator 1 design
- (20 pts) Design by discrete equivalent: Compensator 1 implementation and performance
- (20 pts) Direct digital design: Compensator 2 design
- (20 pts) Direct digital design: Compensator 2 implementation and performance
- (10 pts) Discussion and analysis of results

Don't forget to embed your Arduino and MATLAB code in your lab writeup!